Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Batch: A1 **Roll No.: 16010123012**

Experiment / Assignment / Tutorial No.: 03

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of the Staff In-charge with date

Title: Buffer Overflow Attack

Objective: To understand and implement different types of buffer overflow vulnerabilities and related security flaws through practical coding demonstrations.

Expected Outcome of Experiment:

Course Outcome	After successful completion of the course students should be able to
	Students should be able to identify software vulnerabilities and apply secure coding practices to mitigate security risks.

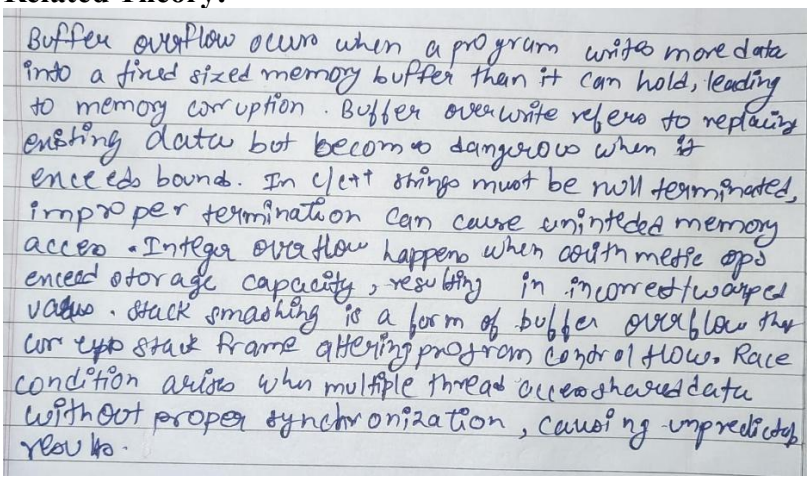
Books/ Journals/ Websites referred:

https://www.geeksforgeeks.org/cpp/buffer-overflow-attack-with-example/

Abstract:

This experiment focuses on understanding buffer overflow and related software vulnerabilities through theoretical discussion and practical implementation. Programs demonstrating buffer overflow, buffer overwrite, integer overflow, null termination issues, stack smashing, and race conditions were implemented and analysed. The experiment highlights how improper memory handling leads to security flaws and emphasizes the importance of secure coding practices to prevent exploitation.

Related Theory:


Buffer overflow occurs when a program writes more data into a fixed sized memory buffer than it can hold, leading to memory corruption. Buffer overwrite refers to replacing existing data but becomes dangerous when it exceeds bounds. In C/C++ strings must be null terminated, improper termination can cause unintended memory access. Integer overflow happens when arithmetic ops exceed storage capacity, resulting in incorrect/warped values. Stack smashing is a form of buffer overflow that corrupts stack frame altering program control flow. Race condition arises when multiple threads access shared data without proper synchronization, causing unpredictable results.

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Implementation Details:

Buffer Overflow:

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char buffer[10];

    cout << "Enter a string: ";
    cin >> buffer;
    cout << "You entered: " << buffer << endl;
    return 0;
}
```

```
Enter a string: adasfgdhcajkchjaschjahs
You entered: adasfgdhcajkchjaschjahs
Segmentation fault


=== Code Exited With Errors ===
```

Buffer Overwrite:

```
#include <iostream>
using namespace std;

int main() {
    char buffer[20];

    cout << "Enter first string: ";
    cin.getline(buffer, 20);
    cout << "Stored value: " << buffer << endl;

    cout << "\nEnter second string: ";
    cin.getline(buffer, 20);
    cout << "After overwrite: " << buffer << endl;

    return 0;
}
```

```
Enter first string: Hello
Stored value: Hello

Enter second string: World
After overwrite: World


=== Code Execution Successful ===
```

Somaiya Vidyavihar University
K. J. Somaiya School of Engineering, Mumbai-77

Null Termination of String:

```
#include <iostream>
using namespace std;

int main() {
    char buffer[20] = "ABCDEFGHIJ";
    char str[5] = {'H','e','l','l','o'};

    cout << str << endl;

    return 0;
}
```

```
HelloABCDEFGHIJ


=== Code Execution Successful ===
```

Integer Overflow:

```
#include <iostream>
#include <climits>
using namespace std;

int main() {
    int x;

    cout << "Enter the number: ";
    cin >> x;

    int y = x + 1;

    cout << y;

    return 0;
}
```

```
Enter the number: 2147483647


-2147483648

=== Code Execution Successful ===
```

Stack Smashing:

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char buffer[10];

    cout << "Enter a string: ";
    cin >> buffer;
```

```
    cout << buffer;

    return 0;
}
```

```
Enter a string: AAAAAAAAAAAAAAAAAAAA

Segmentation fault

=== Code Exited With Errors ===
```

Race Conditions:

```
#include <iostream>
#include <thread>
using namespace std;

int counter = 0;

void increment() {
    for (int i = 0; i < 100000; i++) {
        counter++;
    }
}

int main() {
    thread t1(increment);
    thread t2(increment);

    t1.join();
    t2.join();

    cout << "Final counter value: " << counter << endl;

    return 0;
}
```

```
Final counter value: 138824

=== Code Execution Successful ===
```

```
Final counter value: 129344

=== Code Execution Successful ==
```

```
Final counter value: 150590

=== Code Execution Successful ==
```

Conclusion:

In this experiment, various memory-related vulnerabilities such as buffer overflow, buffer overwrite, null termination issues, integer overflow, stack smashing, and race conditions were implemented and analyzed. The practical demonstrations showed how improper input handling and unsafe memory operations can corrupt program behavior and compromise system security.

Post-Lab Questions

1. Can a format string vulnerability can lead to memory overwrite without

directly using strcpy/gets? Justify your answer with suitable example.

Yes, A format string vulnerability can overwrite memory because functions like printf() interpret user input as formatting instructions. The %n specifier writes the number of printed characters to a memory address, allowing attackers to modify memory without buffer copying functions. Example (C):

```
#include <stdio.h>
int main() {
    int value = 0;
    printf("Before: %d\n", value);

    printf("AAAA%n\n", &value);  // writes 4 into value

    printf("After: %d\n", value);
    return 0;
}
```

Here, %n overwrites value, demonstrating memory modification without strcpy() or gets().

2. Why should programmers guard against unsigned integer overflow even though the C standard defines its behavior?

Although unsigned integer overflow is defined to wrap around modulo the maximum value, it can still cause serious security issues:

- Incorrect memory allocation sizes
- Logic errors in loops or conditions
- Bypass of boundary checks
- Potential buffer overflows due to under-allocation

Attackers can exploit predictable wraparound behavior to manipulate program logic and trigger vulnerabilities.

3. Write a C code snippet demonstrating the TOCTOU vulnerability.

TOCTOU (Time Of Check To Time Of Use) occurs when a resource is checked and later used, allowing changes in between. Example (C):

```
#include <stdio.h>
#include <unistd.h>
int main() {
    const char *filename = "data.txt";
    if (access(filename, W_OK) == 0) {    // Check permission
        sleep(5);                         // Attacker can modify file here
        FILE *fp = fopen(filename, "w"); // Use resource
        if (fp) {
            fprintf(fp, "Writing data\n");
            fclose(fp);
        }
    }
    return 0;
}
```

Between the check and use, the file could be replaced or permissions changed, leading to unintended behavior.